

CIR 策略问题分析与最小修改说明

1. 结论

原始 CIR 策略效果不好的主要原因不是 CIR 数值计算错误，也不是缺少严格的 PIR Gate，而是 **CIR 的分配顺序使用了 FCFS**。

在一块 SSD 的 CIR 总容量不足以同时满足所有活跃 demand 时，原策略按到达时间依次分配：先到的 demand 先拿满 requested CIR，后到的 demand 只能拿剩余 CIR，甚至拿到 0。到达时间与 demand 大小、剩余工作量以及哪个 demand 能更快解除 GPU 的存储等待没有必然关系。因此，一个稍早到达的大 demand 可能长期占用 CIR，而许多本可以很快完成的小 demand 被压到 EXCESS 路径，导致更多 GPU 同时等待 IO。

本次真正带来性能提升的核心改动很小：

原策略排序键：(arrival_time, arrival_order)

新策略排序键：(batch_total_bytes, arrival_time, arrival_order)

也就是在每块 SSD 上优先给完整 logical demand/coflow 更小的任务分配 CIR；大小相同时仍用 FCFS，保证结果确定。该策略不读取 deadline，不计算 Utility，也不是 EDF。

客户端“每批最多 8 个 IO”以及 submission_complete final 标志也已实现，但它们主要解决分批下发时的 demand 生命周期正确性。实验表明，batch=8 本身不是这次性能提升的来源，因此默认关闭。

2. Baseline 和原始 CIR 的实际区别

2.1 Baseline

Baseline 不创建 demand-aware rate controller。QoS 使用配置中的静态 Queue CIR、静态两级 WRR 权重以及 uncapped PIR：

- 各 Queue 有相对稳定的静态服务份额；
- 多个 GPU 的 Queue 可以交错推进；
- 没有根据当前 logical demand 动态重排 CIR。

这种方式不一定最小化单个 demand 的完成时间，但不容易因为某个早到的大 demand 而让后续大量 Queue 长时间失去稳定服务。

2.2 原始 CIR

原始 DemandAwareFCFSCIRController 在每块 SSD 上独立维护活跃 demand，并执行：

remaining = SSD_capacity

```
for demand in active_demands ordered by FCFS:
    assigned_cir[demand] = min(requested_cir[demand], remaining)
    remaining -= assigned_cir[demand]
```

Queue 的 PIR 仍为 uncapped。QoS 的仲裁顺序是 CIR-first：

1. 只要全局存在 Queue 同时满足“非空、CIR token 足够、PIR 允许”，就在 CIR 候选中执行分层 WRR；
2. 只有不存在任何 CIR 候选时，才进入 EXCESS 轮；
3. CIR=0 的 Queue 因为 PIR uncapped 仍然能发送，但只能等待 EXCESS 机会。

所以，assigned_cir=0 并不等于完全禁止发送，却意味着该 Queue 没有稳定的 CIR 服务机会。只要前面的 FCFS demand 持续产生可用 CIR token，后到 demand 的推进就可能明显变慢。

3. 原始 CIR 的具体问题

3.1 FCFS 的优先级与目标指标不匹配

当前目标是提高平均 GPU utilization。对于 pipeline workload，IO demand 完成后，GPU 才能解除 storage stall、进入计算并推进后续 layer/inference。

FCFS 只回答 “谁先到”，没有回答：

- 谁的剩余 IO 更少；
- 给谁带宽可以最快释放一张 GPU；
- 哪个 logical demand 可以最快整体完成；
- 哪个选择能减少同时处于 storage stall 的 GPU 数量。

初始 arrival jitter 还会让 FCFS 排名受几十毫秒的随机到达差异影响。早到并不意味着工作量小，也不意味着优先完成它对整体 GPU 利用率更有价值。

3.2 CIR-first 会放大一次错误排序的影响

原始 CIR 不是在所有 Queue 之间平滑地调整一点权重，而是先把有限 CIR 容量按 FCFS 分配给少数 demand。QoS 又始终优先处理 CIR-eligible Queue，因此错误的 FCFS 选择会反复影响后续每次仲裁。

这解释了为什么原始 CIR 有时甚至差于 Baseline。128 GPU 的原始结果中：

SSD 数量	Baseline 平均 GPU 利用率	原始 FCFS-CIR	绝对变化
1	16.091%	15.036%	-1.055pp
2	31.563%	29.665%	-1.898pp
3	38.674%	37.198%	-1.476pp
4	56.551%	57.783%	+1.232pp
5	64.380%	67.881%	+3.501pp
6	73.085%	79.851%	+6.767pp
7	85.564%	94.235%	+8.671pp
8	97.599%	98.752%	+1.153pp

低 SSD 数量时竞争最强，FCFS 错误排序的代价最大；SSD 数量增加后，容量可以同时覆盖更多 demand，排序的重要性下降；到 8 SSD 左右，Baseline 已接近 100%，可提升空间本身就很小。

3.3 每块 SSD 只看本地 FCFS，缺少 logical demand 完成导向

一个 GPU layer 的 IO 可能分布在多块 SSD 上。GPU 等待的是整个 layer，而不是某一条 SSD path：某些 path 提前完成但最慢 path 仍未完成时，GPU 依然不能进入计算。

原策略只保存本 SSD 上的到达时间和 requested CIR，没有使用已经由 KV Placement 提供的完整 batch_total_bytes。因此它没有显式让同一个较短 logical demand 的各 SSD path 获得一致优先级，也没有尽量缩短 coflow 的整体完成时间。

3.4 原始 Queue-empty 生命周期不支持客户端分批

原策略的隐含前提是：一个 demand 的全部 IO 一次性进入 QoS。因此只要控制器观察到对应 Queue depth 变成 0，就会删除 demand、释放 CIR，并把它计为完成。

如果 UCM 将一个 demand 拆成 8 + 8 + ... 多个 chunk，下列状态是合法的：

chunk 0 已离开 QoS Queue

↓

Queue depth 暂时为 0

↓

客户端仍未提交 chunk 1

原控制器会在中间这个空窗期误判 demand 已完成。下一批到达后又被登记成一个新 demand，造成：

- 原始 arrival order 丢失；
- CIR 被提前释放和重新分配；
- 一个 logical demand 被重复计数；
- 分批行为改变控制面语义，而不仅是改变入口窗口。

这是分批下发必须修复的问题，但不是原始非分批实验中 CIR 收益很小的根因。

4. 诊断实验：排除不是主因的因素

为了分离不同因素，在已有 64 GPU、2 SSD trace 上做了消融：

策略	平均 GPU 利用率	相对 Baseline 的绝对变化
Baseline	60.150%	—
原始 FCFS-CIR	63.432%	+3.283pp
FCFS-CIR + 非 CIR PIR Gate	63.449%	+3.299pp
Shortest-CIR	85.809%	+25.659pp
Shortest-CIR + 非 CIR PIR Gate	85.829%	+25.680pp

这个结果说明：

- 给非 CIR Queue 增加严格 PIR Gate 只有约 0.02pp 影响，uncapped EXCESS 不是主因；
- 只替换排序键就增加约 22.38pp，FCFS 才是主要瓶颈；
- 没有必要引入 Utility/EDF 或严格 admission gate 才能获得明显收益。

客户端分批也做了两类实验：

1. **Queue-empty 立即补下一批**：chunk size 为 2、4、8、16、32 时与不分批结果完全一致。原因是 FCP 每次接受一个完整 descriptor 后施加 backpressure；在同一仿真时刻立即补回下一批，会在 SSD 接受下一个 descriptor 前恢复相同的 Queue 候选集合，因而没有改变实际仲裁。
2. **上一批 IO 全部 completion 后补下一批**：确实改变了候选集合，但 batch=8 的提升只有约 +0.97pp，低于不分批 FCFS-CIR 的 +3.28pp。窗口太小还可能让 DPU/SSD 暂时吃不满。

因此，客户端分批被保留为可选能力，而没有作为默认性能策略。

5. 本次修改

5.1 将完整 coflow 大小作为 CIR 第一排序键

控制器新增 ordering：

- fcfs：完全复现原始顺序；
- shortest：按完整 batch_total_bytes 升序，再按到达时间和登记顺序打破平局。

伪代码如下：

```
priority(demand) = (
    demand.batch_total_bytes,
    demand.arrival_time,
    demand.arrival_order,
)

remaining = SSD_capacity
for demand in sorted(active_demands, key=priority):
    assigned_cir[demand] = min(demand.requested_cir, remaining)
    remaining -= assigned_cir[demand]
```

这里使用的是完整 logical layer/coflow 字节数，而不是“当前 chunk 的字节数”。这样即使客户端分批，同一 demand 的优先级也不会随着 chunk 大小变化；同一个 logical demand 在不同 SSD path 上也携带相同的总大小排序信息。

Pipeline 比较入口默认使用 shortest，同时保留 --cir-ordering fcfs 做回归和消融。通用控制器的构造默认值仍是 fcfs，避免无意改变项目其他入口。

5.2 增加客户端 completion-credit 流量编排

ClientTrafficOrchestrator 按 SSD path 拆分一个 logical demand，每个 chunk 最多包含 N 个 IO：

提交 chunk 0 (最多 N 个 IO)



等待 chunk 0 的所有 SSD completion ACK



提交 chunk 1



.....



提交带 submission_complete=true 的最后一个 chunk

使用 --cir-client-io-chunk-size 8 可以启用每批 8 个 IO。该选项只作用于 cir_only，不会改变 Baseline。

5.3 用 final 标志修复 demand 生命周期

每个 chunk 携带：

- submission_chunk_index：从 0 开始的 chunk 序号；
- submission_chunk_count：完整 demand 的 chunk 总数；
- submission_complete：当前 chunk 是否为最后一批。

控制器持续保存一个 logical demand 的首次 arrival/order、完整 batch_total_bytes 和 CIR 请求。中间 chunk 排空时只设置 awaiting_next_chunk=true，不删除 demand；只有满足以下两个条件才释放：

submission_complete == true

AND

QoS Queue depth == 0

控制器还拒绝乱序 chunk、变化的 chunk 总数、变化的 demand ID、变化的 requested CIR，以及 final 标志与 chunk 序号不一致等非法状态。

6. 为什么 Shortest-CIR 会明显变好：一个例子

假设某块 SSD 的总 CIR 容量为 40 GB/s，同时有三个 demand，每个都请求 25 GB/s：

Demand	到达时刻	完整 coflow 大小	requested CIR
A	0 us	8 GB	25 GB/s
B	1 us	1 GB	25 GB/s
C	2 us	1 GB	25 GB/s

原始 FCFS-CIR 的第一次分配是：

A = 25 GB/s

B = 15 GB/s

C = 0 GB/s

A 很大，却因为只早到 1~2 us 而先拿满 CIR。B、C 本来很快就可以完成并解除两张 GPU 的 storage stall，现在 B 变慢、C 主要等待 EXCESS。即使 A 的一个 SSD path 较早推进，只要 A 在其他 SSD 上的 path 尚未完成，对应 GPU 仍不能计算。

Shortest-CIR 的第一次分配是：

B = 25 GB/s

C = 15 GB/s

A = 0 GB/s

B 很快完成后释放 25 GB/s，C 随后拿满 CIR；B、C 对应的 GPU 更早进入计算。A 虽然推迟，但系统在相同时间内完成了更多 logical demand，减少了平均 storage stall，并让后续计算和下一层预取更早进入 pipeline。对于“平均 GPU utilization”这一目标，这通常比让一个大 demand 较早取得部分进展更有效。

其本质与经典 shortest-processing-time 的效果相同：在资源竞争强、任务大小差异明显时，优先完成短工作会降低平均完成时间。不过当前实现没有引入 deadline，也没有 Utility/EDF 的动态抢占逻辑。

7. 128 GPU 验证结果

新一轮实验使用 128 GPU、3 个 inference、4 个 layer、50 ms 初始 jitter，客户端分批关闭，唯一关键策略变化为 `cir_ordering=shortest`。

SSD 数量	Baseline 平均 GPU 利用率	Shortest-CIR	绝对变化
2	31.563%	77.057%	+45.493pp
4	56.551%	85.882%	+29.331pp
8	97.599%	97.854%	+0.256pp
12	99.615%	99.368%	-0.247pp
14	99.964%	99.811%	-0.152pp
16	100.000%	99.895%	-0.105pp
20	100.000%	99.902%	-0.098pp

SSD=2 和 SSD=4 的平均 TTFT 也同时下降：

SSD 数量	Baseline 平均 TTFT	Shortest-CIR 平均 TTFT
2	890.762 ms	545.572 ms
4	476.180 ms	364.114 ms

8 SSD 及以上没有 10 个百分点的提升空间：Baseline 在 8 SSD 已为 97.599%，理论上最多只能增加 2.401pp；16/20 SSD 已为 100%。因此，在给定的 {2,4,8,12,14,16,20} 七个点和当前指标下，数学上最多只有 SSD=2、4 两个配置可能实现绝对 +10pp。

8. 收益与代价

Shortest-CIR 明显优化了平均 GPU utilization 和平均 TTFT，但它不是免费的：大 demand 会等待更多时间。

SSD 数量	Baseline P95 TTFT	Shortest-CIR P95 TTFT
2	1118.938 ms	1668.711 ms
4	599.852 ms	807.455 ms

因此当前策略适合“平均 GPU utilization 优先”的目标，但会牺牲一部分尾延迟。如果后续还要求限制 P95/Max TTFT，最小扩展应是给 shortest 排序增加简单 aging，例如 demand 等待超过固定阈值后提升到最高优先级。aging 只使用等待时间和固定阈值，不需要引入 Utility 或 EDF，但预计会降低一部分平均利用率收益，需要单独确定目标和阈值。

9. 代码和实验位置

- CIR 排序与分批 demand 生命周期：[rate_controller.py](#)
- DPU 接收并校验 chunk 元数据：[dispatcher.py](#)
- UCM 上层客户端流量编排和策略参数：[run_pipelined_ucm_comparison.py](#)
- 回归测试：[test_pipelined_ucm_comparison.py](#)
- 原始 FCFS-CIR 结果：[原始实验 summary.json](#)
- Shortest-CIR 结果：[新实验 summary.json](#)
- 利用率数据：[新实验 CSV](#)

相关测试覆盖：

- 中间 chunk 排空不能释放 demand；
- 最后一个 chunk 排空后才能释放 demand；
- 17 个 IO 按 8 + 8 + 1 分批并保持请求/字节守恒；
- shortest 必须使用完整 coflow 字节数，而不是当前 chunk 字节数；
- Pipeline 默认使用 shortest，通用控制器默认仍为 FCFS。